# CSP-203: Software Tools & Technologies Lab




## Chapter – 8

## Debugging Tools: GDB (Lab Session - 23)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

# Contents


- ❑ Introduction
- ❑ Compilation and Run
- ❑ Help Command
- ❑ Exiting Command
- ❑ Executing Your Program
- ❑ Shell Commands
- ❑ Source File Navigation
- ❑ Working with Files
- ❑ Execution Control
- ❑ Breakpoints and Watchpoints
- ❑ Program Stack
- ❑ Display
- ❑ Automatic Display
- ❑ Expressions
- ❑ Symbol Table Inspection
- ❑ GDB scripting
- ❑ Signal Handling
- ❑ Debugging Targets
- ❑ Controlling GDB

**What is Debugger?**

- A debugger is a tool that helps programmers find and fix errors (bugs) in their code. It allows you to:
  - Set Breakpoints: You can pause the execution of your program at specific lines of code.
  - Step Through Code: Execute your code line by line to see exactly what's happening.
  - Step Over: Run the current line and move to the next one.
  - Step Into: Go into a function call to see its inner workings.
  - Step Out: Exit the current function and return to the calling code.
  - Inspect Variables: View the values of variables at different points in execution.
  - Watch Expressions: Track how a specific expression or variable changes over time.
  - Call Stack Inspection: See the list of function calls that led to the current point (the "call stack").

**Debugger (GDB):**

- The GNU Debugger (GDB) is a portable debugger that runs on many Unix-like systems.
- works for many programming languages, including Ada, Assembly, C, C++, D, Fortran, Haskell, Go, Objective-C, OpenCL C, Modula-2, Pascal, Rust,[2] and partially others.

**History:**

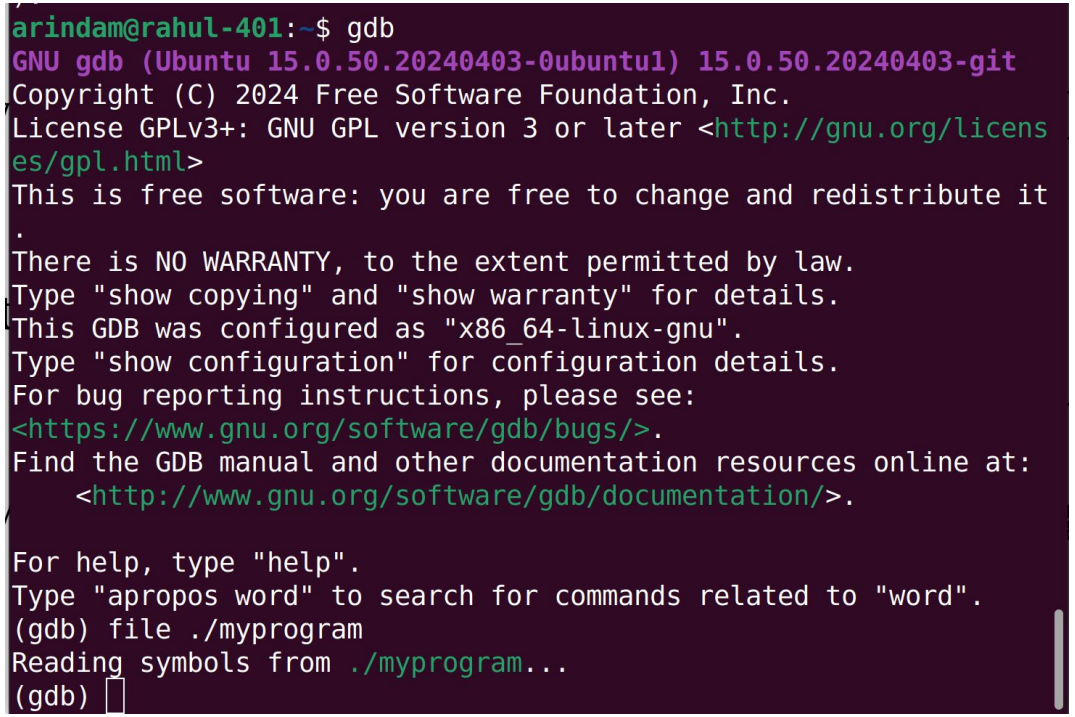
- GDB was first written by Richard Stallman in 1986 as part of his GNU system.
- From 1990 to 1993 it was maintained by John Gilmore.
- Now it is maintained by the GDB Steering Committee which is appointed by the Free Software Foundation.

**Installing GDB in Linux and compiling program:**

- Update packages list:
  - sudo apt update
- Install GDB:
  - sudo apt install gdb
- Verify GDB installation:
  - gdb --version
- Compiling with debug informations:
  - use -g flag with your compiler to include debugging information: gcc -g myprogram.c -o myprogram
- Start GDB with your compiled program:
  - gdb ./myprogram

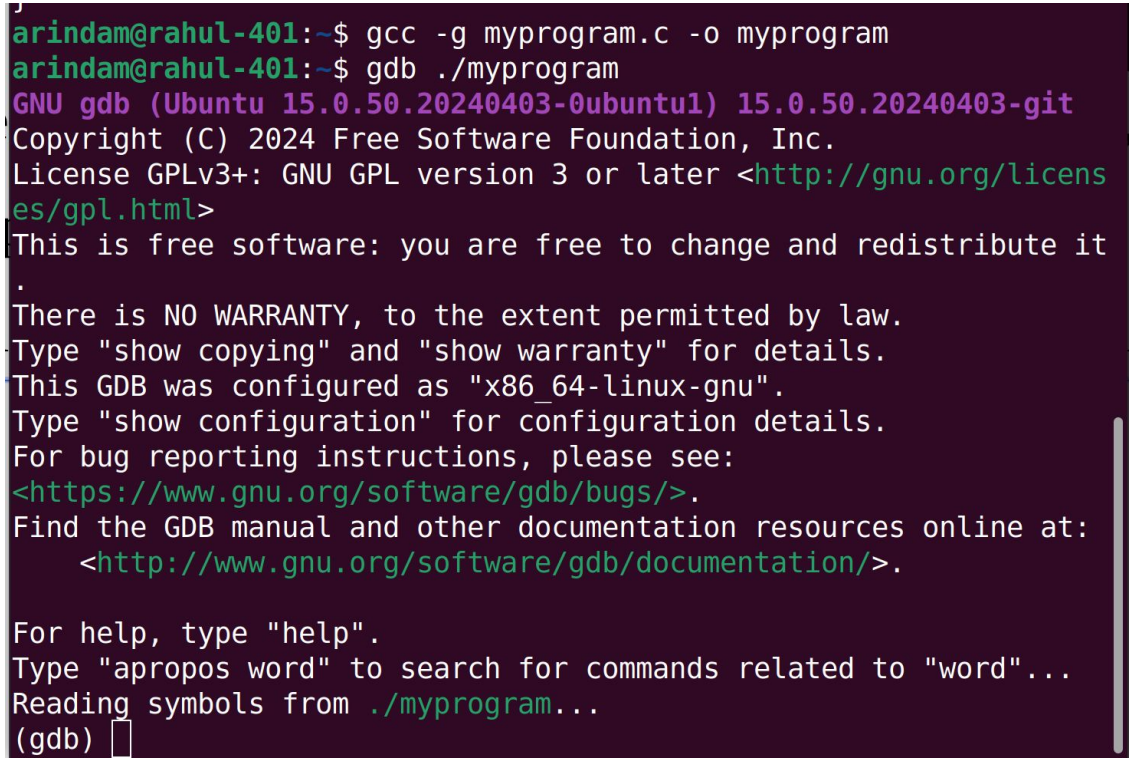
## Starting with GDB:

- gdb
  - Starts GDB without loading any executable. You get the (gdb) prompt, ready for commands. You can use it when:
    - You want to load symbols or binaries later via GDB commands (file ./myprogram).
    - You just want to explore GDB itself or inspect remote targets.



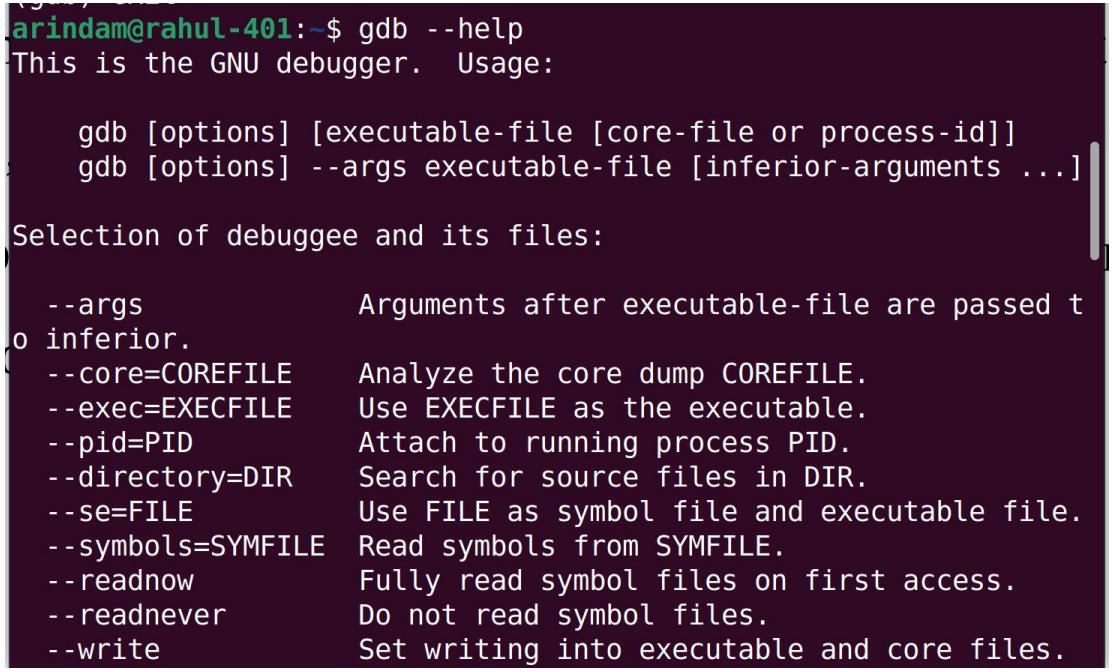
## Starting with GDB:

- gdb program
  - Loads the executable named program (and its debug symbols, if compiled with -g) into GDB and drops you immediately into the GDB prompt.
  - When you want to start debugging a program, you can use this command.



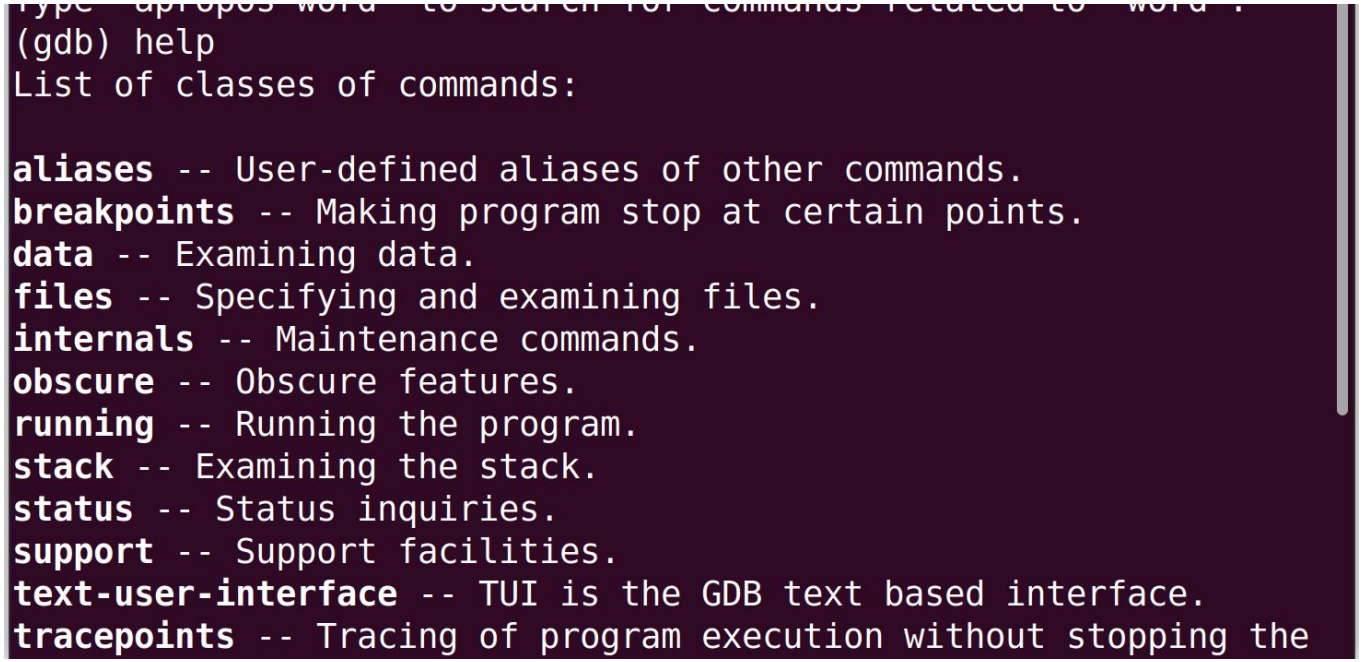
**Help command:**

- gdb --help
  - Prints a summary of the command-line options you can pass to GDB itself (flags like --args, --quiet, --tui, etc.), then exits immediately.
  - If you need to recall or discover startup flags without diving into the full manual, you can use this command.



## Help command:

- help
  - shows the top-level categories (classes) of GDB commands.



## Help command:

- help <class>
  - Lists all commands in the given class, with a one-line summary each.

```
(gdb) help breakpoints
Making program stop at certain points.

List of commands:

awatch -- Set an access watchpoint for EXPRESSION.
break, brea, bre, br, b -- Set breakpoint at specified location.
break-range -- Set a breakpoint for an address range.
catch -- Set catchpoints to catch events.
catch assert -- Catch failed Ada assertions, when raised.
catch catch -- Catch an exception, when caught.
catch exception -- Catch Ada exceptions, when raised.
catch exec -- Catch calls to exec.
catch fork -- Catch calls to fork.
catch handlers -- Catch Ada exceptions, when handled.
catch load -- Catch loads of shared libraries.
catch rethrow -- Catch an exception, when rethrown.
catch signal -- Catch signals by their names and/or numbers.
catch syscall -- Catch system calls by their names, groups and/or
 numbers.
catch throw -- Catch an exception, when thrown.
catch unload -- Catch unloads of shared libraries.
```

## Help command:

- help <command>
  - Lists all commands in the given class, with a one-line summary each.

```
(gdb) help break
break, brea, bre, br, b
Set breakpoint at specified location.
break [PROBE_MODIFIER] [LOCATION] [thread THREADNUM]
        [-force-condition] [if CONDITION]
PROBE_MODIFIER shall be present if the command is to be placed in
 a
probe point.  Accepted values are `-probe' (for a generic, automa
tically
guessed probe type), `-probe-stap' (for a SystemTap probe) or
`-probe-dtrace' (for a DTrace probe).
LOCATION may be a linespec, address, or explicit location as desc
ribed
below.

With no LOCATION, uses current execution address of the selected
stack frame.  This is useful for breaking on return to a stack fr
ame.
```

**Stopping GDB:**

- “quit” or “q” command:
  - Exits the GDB.
- Press Ctrl +D at the (gdb) prompt. This sends an end-of-file and GDB will exit immediately (no confirmation if nothing’s running).
- “kill” then “quit” command:
  - This kills the inferior process first, then exits GDB.
- interrupt (or Ctrl +C):
  - If you’re typing a GDB command that’s taking too long or hung, it stops that GDB command and returns you to the (gdb) prompt.



## Executing your program with GDB:

- run [arglist]
  - Starts your program under GDB, passing it the given arguments. If you omit arglist, GDB uses whatever arguments were last set (via run or set args).
  - Syntax: run [arg1 arg2 …]

```
(gdb) run 3 4
Starting program: /home/arindam/sum 3 4

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Sum = 7
[Inferior 1 (process 202761) exited with code 010]
(gdb)
```

## Executing your program with GDB:

- Redirecting I/O: run … < infile > outfile
  - Redirects the program's stdin and stdout just like in the shell.
  - Syntax: (gdb) run < test.in > test.out

```
arindam@rahul-401:~$ cat sum.c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a, b;
    while (scanf("%d %d", &a, &b) == 2) {
        printf("%d + %d = %d\n", a, b, a + b);
    }
    return 0;
}
```

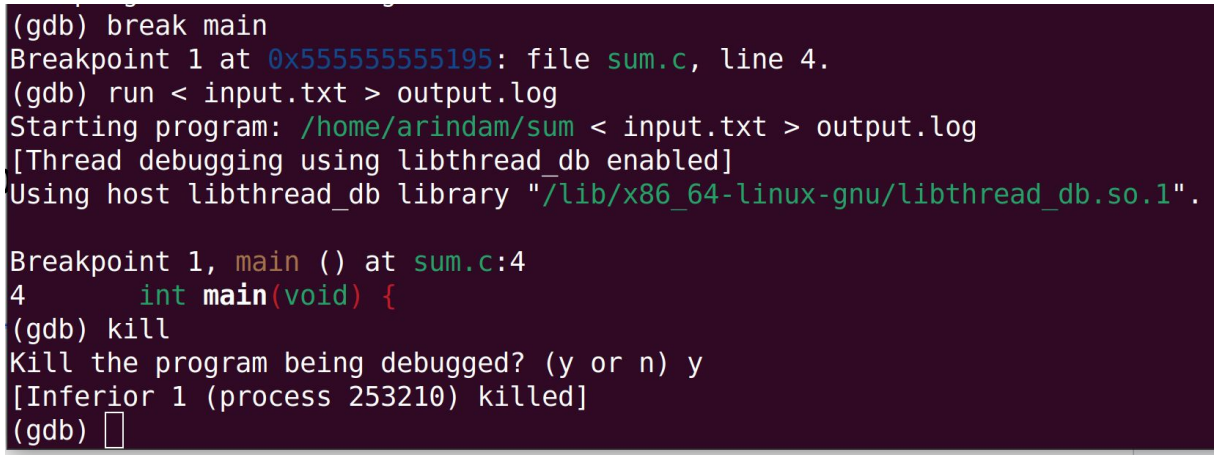
```
arindam@rahul-401:~$ cat input.txt
1 3
2 4
4 5
arindam@rahul-401:~$ gdb ./sum
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sum...
(gdb) run < input.txt > output.log
Starting program: /home/arindam/sum < input.txt > output.log

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[Inferior 1 (process 242857) exited normally]
(gdb) exit
arindam@rahul-401:~$ cat output.log
1 + 3 = 4
2 + 4 = 6
4 + 5 = 9
arindam@rahul-401:~$
```

## Executing your program with GDB:

- run (no args)
  - Exactly the same as above, but uses the current argument list. Good for fast “rerun” when you’re tweaking code.
  - (gdb) run # re-runs the last invocation, e.g. `./sum < input.txt > output.log`
- kill
  - Terminates the currently running inferior (your debugged process) so you can start again with run.



## Executing your program with GDB:

- tty <dev>
  - On the next run, directs your program's stdin and stdout to the given terminal device (e.g. another xterm or serial port).

```
arindam@rahul-401:~$ cat mul.c
#include<stdio.h>
void main(){
        printf("Enter the two numbers: ");
        int a,b;
        scanf("%d %d",&a,&b);
        printf("The multiplication is: %d",a*b);
}
arindam@rahul-401:~$
```

```
Reading symbols from ./mul...
(gdb) tty /dev/pts/1
(gdb) run
Starting program: /home/arindam/mul

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[Inferior 1 (process 316089) exited normally]
(gdb)
```

```
arindam@rahul-401: ~
arindam@rahul-401:~$ tty
/dev/pts/1
arindam@rahul-401:~$ warning: GDB: Failed to set con
Enter the two numbers: The multiplication is: 20
```

## Executing your program with GDB:

- set args [arglist]
  - Pre-sets the argument list for future run commands. Saves you from retyping it every time.
  - Syntax: set args arg1 arg2 …

```
arindam@rahul-401: ~
arindam@rahul-401:~$ cat sum_args.c
#include<stdio.h>
#include<stdlib.h>

void main(int argc, char *argv[]){
        int a = atoi(argv[1]);
        int b = atoi(argv[2]);
        printf("The sum of %d + %d is %d\n",a,b,(a+b));
}
arindam@rahul-401:~$
```

```
Reading symbols from ./sum_args...
(gdb) set args 5 7
(gdb) run
Starting program: /home/arindam/sum_args 5 7

This GDB supports auto-downloading debuginfo from the follo
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled
dbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libt
o.1".
The sum of 5 + 7 is 12
[Inferior 1 (process 480746) exited with code 027]
(gdb)
```

**Executing your program with GDB:**

- show args
  - Displays the current argument list that GDB will pass on run.
  - Syntax: show args

```
(gdb) show args
Argument list to give program being debugged when it is started is "5
 7".
(gdb)
```

## Executing your program with GDB:

- set args
  - Specify empty argument list.
  - Syntax: set args

```
(gdb) set args
(gdb) show args
Argument list to give program being debugged when it is started is ""
.
(gdb)
```

## Executing your program with GDB:

- show env
  - Without a variable: lists all environment variables GDB will pass to your program.
  - With a variable: shows the value of that specific variable.
  - Syntax:

    show env  # list all

    show env VAR_NAME  # show just one

```
(gdb) show env
SHELL=/bin/bash
SESSION_MANAGER=local/rahul-401:@/tmp/.ICE-unix/410091,uni
:/tmp/.ICE-unix/410091
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
LANGUAGE=en_IN:en
```

```
(gdb) show env HOME
HOME = /home/arindam
(gdb)
```

**Executing your program with GDB:**

- set env <var> <string>
  - Defines or changes the environment variable <var> to <string> for your program's process.
  - Syntax: set env VAR_NAME value [Important: Not to perform arbitrarily]
- unset env <var>
  - Removes <var> from the environment that GDB passes to your program.
  - Syntax: unset env VAR_NAME [Important: Not to perform arbitrarily]

**Shell Commands with GDB:**

- cd dir
  - Changes GDB's current working directory to dir. This affects:
    - File paths used by GDB (e.g., for loading symbols, opening core dumps).
    - The working directory used when your program runs (i.e., where main() runs from).

```
(gdb) cd dir1
Working directory /home/arindam/dir1.
```

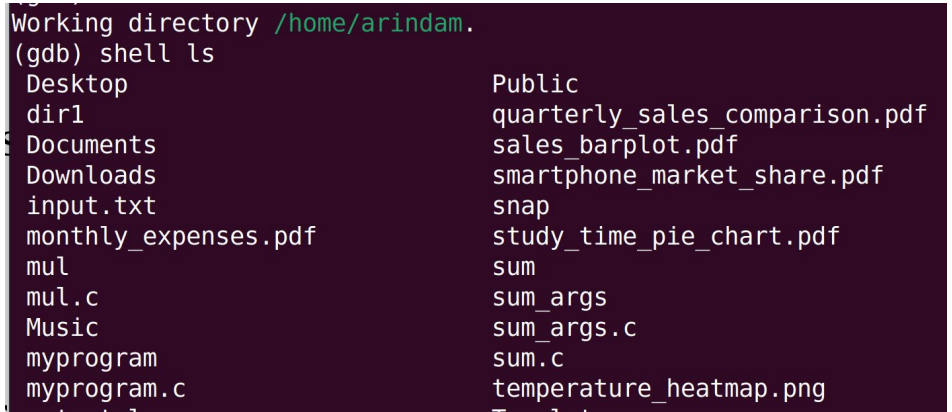
- pwd
  - Prints GDB's current working directory.

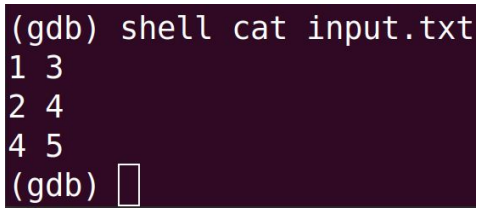
```
(gdb) pwd
Working directory /home/arindam/dir1.
```

- cd ..
  - Returns to parent directory.

## Shell Commands with GDB:

- make
  - (gdb) make : This runs the default make target (like running make in the terminal). It's useful for quickly rebuilding from within GDB.
  - (gdb) make clean, (gdb) make debug :GDB will call the system's make clean or make debug.
- shell cmd
  - Runs an arbitrary shell command from inside GDB. Acts like temporarily dropping to a terminal shell.



**Source Files:**

- dir [directory]
  - Syntax: dir /home/user/project/src : This tells GDB to look for source files in /home/user/project/src if it can't find them in the current directory.
- dir (with no args)
  - Clears the source path. GDB forgets all additional directories.
  - Syntax: (gdb) dir
- show dir
  - Displays the current list of directories in the source path.
  - Syntax: (gdb) show dir

```
(gdb) dir /home/arindam/dir1
Source directories searched: /home/arindam/dir1:$cdir:$cwd
(gdb) show directories
Source directories searched: /home/arindam/dir1:$cdir:$cwd
(gdb) dir
Reinitialize source path to empty? (y or n) y
Source directories searched: $cdir:$cwd
(gdb) show directories
Source directories searched: $cdir:$cwd
(gdb)
```

**Source Files:**

- list (or l)
  - Shows source code lines. Default is 10 lines around the current point of execution.
  - Show current source line: (gdb) list
  - Show specific source line: (gdb) list <line_number>
  - Show a range of lines: (gdb) list <start_line>,<end_line>
  - Show start of a function: (gdb) list <func_name>
  - Show from file->line: (gdb) list <file_name>:<line_number> [It will work on the file whose symbol table has been loaded by “file” command in gdb.
  - Reach to current source line: (gdb) list .
  - Show previous ten lines: list -
  - Show lines relative to the last ones printed: list +<offset> or list -<offset>
  - Shows the line of code corresponding to a memory address: list *address

**Source Files:**

- info line [num]
  - Shows the memory address range of the machine code for a source line.
  - Syntax: (gdb) info line <line_number>

```
(gdb) info line 9
Line 9 of "fib.c" starts at address 0x1208 <main>
   and ends at 0x1214 <main+12>.
(gdb)
```

- info source
  - Shows the name and path of the currently loaded source file.
  - Syntax: (gdb) info source

```
(gdb) info source
Current source file is fib.c
Compilation directory is /home/arindam
Located in /home/arindam/fib.c
Contains 23 lines.
Source language is c.
Producer is GNU C17 13.3.0 -mtune=generic -march=x8
chronous-unwind-tables -fstack-protector-strong -fs
tection -fcf-protection.
Compiled with DWARF 5 debugging format.
Does not include preprocessor macro info.
(gdb)
```

**Source Files:**

- info sources
  - Lists all source files GDB is using for this debugging session.
  - Syntax: (gdb) info sources

```
(gdb) info sources
/home/arindam/fib:

/home/arindam/fib.c, /usr/include/stdio.h

(gdb)
```

- forw regex
  - Search forward through source code for lines matching a regex.
  - Syntax: forw <regex_info>

```
(gdb) forw printf
12          printf("Enter the number of terms: ");
(gdb)
```

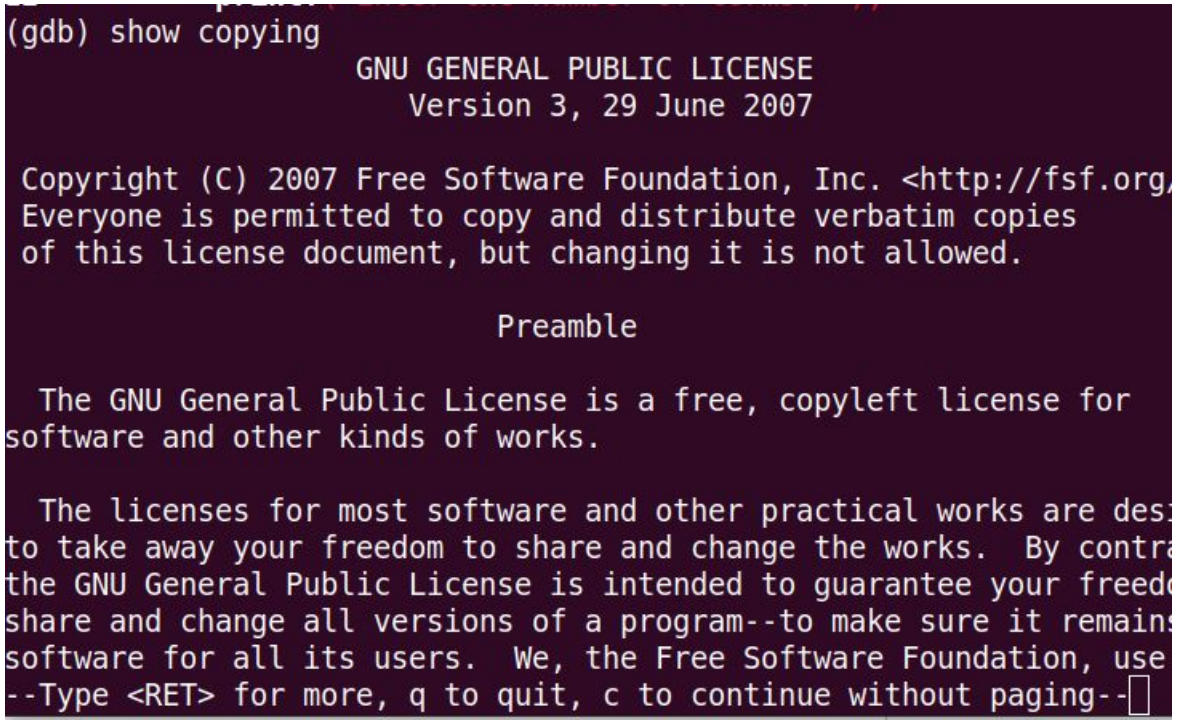
**Source Files:**

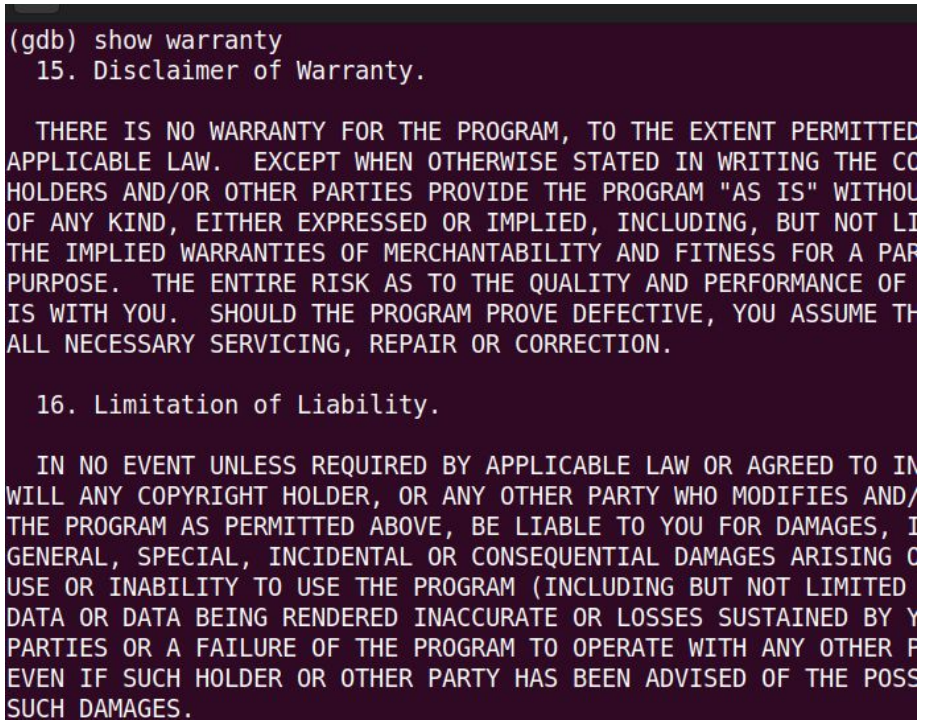
- rev regex
  - Search backward through source code for lines matching a regex.
  - Syntax: (gdb) rev <regex_info>

```
(gdb) forw printf
12          printf("Enter the number of terms: ");
(gdb) forw printf
15          printf("Fibonacci Series:\n");
(gdb) forw printf
17              printf("%d ", fibonacci(i));
(gdb) forw printf
20          printf("\n");
(gdb) forw printf
Expression not found
(gdb) rev printf
17              printf("%d ", fibonacci(i));
(gdb) rev printf
15          printf("Fibonacci Series:\n");
(gdb) rev printf
12          printf("Enter the number of terms: ");
(gdb)
```

## GDB License View:

- show copying
  - Display GNU general public license.
- show warranty
  - There is NO WARRANTY for GDB. Display full no-warranty statement.



**Working Files:**

- file [filename]
  - Loads the given executable and its symbol table. With no argument, it unloads the currently loaded executable and symbols.
  - Syntax: (gdb) file <executable_file>
- core [filename]
  - Load a core dump file for post-mortem debugging.
  - Syntax: (gdb) core <core_dump_file>
  - You can now inspect the state (e.g., bt, info registers) from when the crash occurred.
- exec [filename]
  - Use this file as the executable image only, without loading debug symbols from it.
  - Syntax: (gdb) exe <file_name>
  - Useful if you plan to load symbols separately

**Working Files:**

- symbol [filename]
  - Load only the symbol table (debug info) from a separate file.
  - Syntax: (gdb) symbol <debug_file_name>
- load [filename]
  - Load a file into the target memory (like loading a binary to an embedded target). Usually used with remote debugging or embedded systems.
  - Syntax: (gdb) load <file_name>
- add-symbol-file file addr
  - Adds a symbol file (e.g., for a shared library or module) loaded at a specific memory address.
  - Example: (gdb) add-symbol-file ./libplugin.so 0x600000

**Working Files:**

- info files
  - Shows which files GDB is using — executable, symbol, core dump, and memory mappings.
- path [dirs]
  - Adds directories to the front of the path GDB uses to look for executables and symbols.
- show path
  - Shows the current list of directories GDB searches for binaries and symbols.
- info shared (or info sharedlibrary)
  - Lists all currently loaded shared libraries, their addresses, and symbol status.

**Execution Control:**

- continue [count] or c
  - Resumes program execution until the next breakpoint or the end. If count is given, it skips that breakpoint that many times.
  - Example: (gdb) continue # Continue until fibonacci is hit
    (gdb) continue 2 # Skip this breakpoint the next 2 times it's hit
- step [count] or s
  - Steps into the next line of code, entering function calls.
  - Example: (gdb) step # Enters function() when called
- next [count] or n
  - Steps over the next line of code; doesn’t enter functions.
  - Example: (gdb) next # Skips over function call, moves to next line

**Execution Control:**

- stepi [count] or si
  - Steps by one machine instruction.
  - Syntax: (gdb) stepi
- nexti [count] or ni
  - Steps over one machine instruction, including calls.
  - Syntax: (gdb) nexti
- until [location]
  - Runs until the current function returns to a line after the current one, or a specific location.
  - Example: (gdb) until # Useful in loops, to quickly skip to the next iteration

    (gdb) until 15 # Run until line 15

**Execution Control:**

- finish
  - Runs until the current function returns and shows the return value.
  - Syntax: (gdb) finish
- return [expr]
  - Forces a function to return early, optionally with a custom value.
  - Example: (gdb) return 99 # Forces function to return 99
    (gdb) finish # Finish the stack
- signal num
  - Sends a signal to the program.
  - Example: (gdb) signal 0 # Resume execution with no signal
    (gdb) signal 2 # Send SIGINT (like Ctrl+C)

**Execution Control:**

- jump line or jump *address
  - Skips code and jumps to a line or instruction address.
  - Example: (gdb) jump 12 # Skip to line 12 in current function
- set var=expr
  - Change the value of a variable while running.
  - Example: (gdb) print n # Output: $1 = 5

    (gdb) set n = 10 # Now n is 10

    (gdb) print n # Output: $2 = 10

**Execution Control:**

- jump line or jump *address
  - Skips code and jumps to a line or instruction address.
  - Example: (gdb) jump 12 # Skip to line 12 in current function
- set var=expr
  - Change the value of a variable while running.
  - Example: (gdb) print n # Output: $1 = 5

    (gdb) set n = 10 # Now n is 10

    (gdb) print n # Output: $2 = 10

# Execution Control:

```
Load new symbol table from "./fib"? (y or n) y
Reading symbols from ./fib...
(gdb) break main
Note: breakpoints 2 and 3 also set at pc 0x1214.
Breakpoint 4 at 0x1214: file fib.c, line 9.
(gdb) run
Starting program: /home/arindam/fib
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linu
so.1".

Breakpoint 2, main () at fib.c:9
9       int main() {
(gdb) continue
Continuing.
Enter the number of terms: 5
Fibonacci Series:
0 1 1 2 3
[Inferior 1 (process 2146061) exited normally]
(gdb)
```

```
[Inferior 1 (process 2146061) exited normally]
(gdb) file ./fib
Load new symbol table from "./fib"? (y or n) y
Reading symbols from ./fib...
(gdb) break main
Note: breakpoints 2, 3 and 4 also set at pc 0x1214.
Breakpoint 5 at 0x1214: file fib.c, line 9.
(gdb) run
Starting program: /home/arindam/fib
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-g
so.1".

Breakpoint 2, main () at fib.c:9
9       int main() {
(gdb) next
12          printf("Enter the number of terms: ");
(gdb) next
13          scanf("%d", &n);
(gdb)
```

```
(gdb) next
main () at fib.c:16
16          for (i = 0; i < n; i++) {
(gdb) next
17              printf("%d ", fibonacci(i));
(gdb) step

Breakpoint 1, fibonacci (n=1) at fib.c:3
3           if (n <= 1)
(gdb)
```

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./fib...
(gdb) break main
Breakpoint 1 at 0x1214: file fib.c, line 9.
(gdb) break fibonacci
Breakpoint 2 at 0x11d9: file fib.c, line 3.
(gdb) run
Starting program: /home/arindam/fib

This GDB supports auto-downloading debuginfo from the following
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on'
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread

Breakpoint 1, main () at fib.c:9
9       int main() {
(gdb) continue
Continuing.
Enter the number of terms: 5
Fibonacci Series:

Breakpoint 2, fibonacci (n=0) at fib.c:3
3           if (n <= 1)
(gdb) nexti
0x00005555555551dd      3           if (n <= 1)
(gdb) stepi
4               return n;
(gdb) step
7       }
(gdb) next
main () at fib.c:16
16          for (i = 0; i < n; i++) {
(gdb) until 17
```

```
(gdb) print n
$4 = 2
(gdb) set var n = 10
(gdb) print n
$5 = 10
(gdb)
```

```
(gdb) until 17
main () at fib.c:17
17              printf("%d ", fibonacci(i));
(gdb) continue
Continuing.

Breakpoint 2, fibonacci (n=1) at fib.c:3
3           if (n <= 1)
(gdb) finish
Run till exit from #0  fibonacci (n=1) at fib.c:3
0x0000555555555274 in main () at fib.c:17
17              printf("%d ", fibonacci(i));
Value returned is $1 = 1
(gdb) continue
Continuing.

Breakpoint 2, fibonacci (n=2) at fib.c:3
3           if (n <= 1)
(gdb) return 99
Make fibonacci return now? (y or n) y
#0  0x0000555555555274 in main () at fib.c:17
17              printf("%d ", fibonacci(i));
(gdb) finish
"finish" not meaningful in the outermost frame.
(gdb) continue
Continuing.

Breakpoint 2, fibonacci (n=3) at fib.c:3
3           if (n <= 1)
(gdb) finish
Run till exit from #0  fibonacci (n=3) at fib.c:3

Breakpoint 2, fibonacci (n=2) at fib.c:3
3           if (n <= 1)
(gdb) watch n
Hardware watchpoint 3: n
(gdb) print n
```

# Execution Control:



**Setting breakpoints:**

- break [file:]line
  - Set a breakpoint at a specific line in a file. Syntax: break <filename>:<line_num>
- break [file:]function
  - Set a breakpoint at a function entry point. Syntax: break <function_name>
- break +offset / break -offset
  - Set a breakpoint relative to the current line. Syntax: break <offset_value>
- break *address
  - Set a breakpoint at a specific memory address. Syntax: break *<address>
- Sets a breakpoint at the next instruction (good during stepping). Syntax: break

**Conditional breakpoints:**

- break ... if expr
  - Set a breakpoint that triggers only when the condition is true.
  - Example: (gdb) break main if argc > 2 [Break main if argc > 2]
- cond n expr
  - Attach/change a condition on breakpoint number n.
  - Example: (gdb) cond 2 i == 10 [Makes breakpoint 2 conditional on i == 10]

**Temporary breakpoints:**

- tbreak …
  - Break once and auto-disable.
  - Example: (gdb) tbreak fibonacci1 [Stops at myfunc() once and then deletes the breakpoint]

**Regex breakpoints:**

- rbreak regex
  - Set breakpoints on all functions matching a regex.
  - Example: (gdb) rbreak ^fib [Sets breakpoints on all functions starting with foo]

**Watchpoints:**

- watch expr
  - Breaks when the value of an expression changes.
  - Example: (gdb) watch count [Breaks when count changes]

**Catchpoints:**

- catch event
  - Breaks on events like throw, catch, fork, exec, etc.
  - Example: (gdb) catch throw [Breaks when an exception is thrown (C++)]

**Managing Breakpoints & Watchpoints**

- info break
  - List all breakpoints. Syntax: (gdb) info break
- info watch
  - List all watchpoints. Syntax: (gdb) info watch
- clear [file:]function
  - Delete breakpoints at the function. Syntax: (gdb) clear <function_name>
- delete [n]
  - Delete breakpoint(s) by number or all if omitted. Syntax: (gdb) delete <bp_num>
- disable [n] / enable [n]
  - Disable or enable breakpoints. Syntax: (gdb) enable <bp_num> / disable <bp_num>
- enable once [n]
  - Enable a breakpoint for one hit, then disable. Syntax: (gdb) enable once <bp_num>

## Managing Breakpoints & Watchpoints

- enable delete [n]
  - Enable, then delete after hitting once. Syntax: (gdb) enable delete <bp_num>
- ignore n count
  - Ignore breakpoint n for count times. Syntax: (gdb) ignore <bp_num> <count_number>

## Example Flow:

```
arindam@rahul-401:~$ cat fib.c
#include <stdio.h>
int fibonacci(int n) {
    if (n <= 1)
        return n;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n, i;

    printf("Enter the number of terms: ");
    scanf("%d", &n);

    printf("Fibonacci Series:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", fibonacci(i));
    }

    printf("\n");
    return 0;
}

arindam@rahul-401:~$
```

```
Reading symbols from ./fib...
(gdb) file ./fib
A program is being debugged already.
Are you sure you want to change the file? (y or n) y
Load new symbol table from "./fib"? (y or n) y
Reading symbols from ./fib...
(gdb) break 17
Breakpoint 4 at 0x55555555526a: file fib.c, line 17.
(gdb) break 20
Breakpoint 5 at 0x555555555296: file fib.c, line 20.
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/arindam/fib
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter the number of terms: 10
Fibonacci Series:

Breakpoint 4, main () at fib.c:17
17              printf("%d ", fibonacci(i));
(gdb) ignore 4 8
Will ignore next 8 crossings of breakpoint 4.
(gdb) continue
Continuing.

Breakpoint 4, main () at fib.c:17
17              printf("%d ", fibonacci(i));
(gdb) continue
Continuing.

Breakpoint 5, main () at fib.c:20
20          printf("\n");
(gdb) continue
Continuing.
0 1 1 2 3 5 8 13 21 34
[Inferior 1 (process 668065) exited normally]
(gdb)
```

```
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthre
Enter the number of terms: 5
Fibonacci Series:

Breakpoint 1, main () at fib.c:17
17              printf("%d ", fibonacci(i));
(gdb) watch i
Hardware watchpoint 2: i
(gdb) continue
Continuing.

Hardware watchpoint 2: i

Old value = 0
New value = 1
0x000055555555528e in main () at fib.c:16
16          for (i = 0; i < n; i++) {
(gdb) continue
Continuing.

Breakpoint 1, main () at fib.c:17
17              printf("%d ", fibonacci(i));
(gdb) disable
(gdb) continue
Continuing.
0 1 1 2 3
Watchpoint 2 deleted because the program has left the block
in which its expression is valid.
[Inferior 1 (process 668271) exited normally]
(gdb) info watchpoint
No watchpoints.
(gdb)
```

```
Load new symbol table from "./fib"? (y or n) y
Reading symbols from ./fib...
(gdb) break main
Breakpoint 5 at 0x555555555214: file fib.c, line 9.
(gdb) break fibonacci
Note: breakpoint 3 also set at pc 0x5555555551d9.
Breakpoint 6 at 0x5555555551d9: file fib.c, line 3.
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/arindam/fib
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 5, main () at fib.c:9
9       int main() {
(gdb) watch n
Hardware watchpoint 7: n
(gdb) continue
Continuing.
Enter the number of terms: 5
Download failed: Invalid argument.  Continuing without source file ./stdio

Hardware watchpoint 7: n

Old value = -8704
New value = 5
0x00007ffff7c70148 in __vfscanf_internal (s=<optimized out>, format=<optim
    at ./stdio-common/vfscanf-internal.c:1976
warning: 1976   ./stdio-common/vfscanf-internal.c: No such file or directo
(gdb) continue
Continuing.
Fibonacci Series:

Breakpoint 3, fibonacci (n=0) at fib.c:3
3           if (n <= 1)
(gdb) continue
Continuing.

Breakpoint 3, fibonacci (n=1) at fib.c:3
3           if (n <= 1)
```

```
Old value = -8704
New value = 5
0x00007ffff7c70148 in __vfscanf_internal (s=<optimized out>
    at ./stdio-common/vfscanf-internal.c:1976
warning: 1976   ./stdio-common/vfscanf-internal.c: No such
(gdb) continue
Continuing.
Fibonacci Series:

Breakpoint 3, fibonacci (n=0) at fib.c:3
3           if (n <= 1)
(gdb) continue
Continuing.

Breakpoint 3, fibonacci (n=1) at fib.c:3
3           if (n <= 1)
(gdb) print n
$1 = 1
(gdb) continue
Continuing.

Breakpoint 3, fibonacci (n=2) at fib.c:3
3           if (n <= 1)
(gdb) continue
Continuing.

Breakpoint 3, fibonacci (n=1) at fib.c:3
3           if (n <= 1)
(gdb) continue
Continuing.

Breakpoint 3, fibonacci (n=0) at fib.c:3
3           if (n <= 1)
(gdb) continue
Continuing.

Breakpoint 3, fibonacci (n=3) at fib.c:3
3           if (n <= 1)
(gdb) print n
$2 = 3
(gdb)
```

```
which its expression is valid.
0x0000555555555274 in main () at fib.c:17
17              printf("%d ", fibonacci(i));
(gdb) print n
$1 = 5
(gdb) next
16          for (i = 0; i < n; i++) {
(gdb) next
17              printf("%d ", fibonacci(i));
(gdb) next

Breakpoint 2, fibonacci (n=1) at fib.c:3
3           if (n <= 1)
(gdb) print n
$2 = 1
(gdb) next
4               return n;
(gdb) info break
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x0000555555555214 in main at fib.c:9
        breakpoint already hit 1 time
2       breakpoint     keep y   0x00005555555551d9 in fibonacci at fib.c:3
        breakpoint already hit 2 times
(gdb) disable 2
(gdb) continue
Continuing.
0 1 1 2 3
[Inferior 1 (process 669598) exited normally]
(gdb) info break
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x0000555555555214 in main at fib.c:9
        breakpoint already hit 1 time
2       breakpoint     keep n   0x00005555555551d9 in fibonacci at fib.c:3
        breakpoint already hit 2 times
(gdb) delete
Delete all breakpoints, watchpoints, tracepoints, and catchpoints? (y or n) y
(gdb) info break
No breakpoints, watchpoints, tracepoints, or catchpoints.
(gdb)
```

**Program Stack:**

- backtrace [n] or bt [n]
  - Shows the current call stack (which functions have been called and in what order).
    - n > 0: Show only n innermost frames.
    - n < 0: Show only n outermost frames.
    - No n: Show full stack.
  - Syntax: (gdb) bt
- frame [n]
  - Selects and shows details of a specific stack frame.
  - Syntax: (gdb) frame <frame_number>
- up [n]
  - Moves up the call stack (toward main).
  - Syntax: (gdb) up <number>

**Program Stack:**

- down [n]
  - Moves down the call stack (toward most recent call).
  - Syntax: (gdb) down <number>
- info frame [addr]
  - Shows low-level info about the currently selected frame or frame at a specific address.
  - Syntax: (gdb) info frame
- info args
  - Displays the function arguments of the currently selected frame.
  - Syntax: (gdb) info argos
- info locals
  - Shows the local variables in the selected frame.
  - Syntax: (gdb) info locals

**Program Stack:**

- info reg [register]
  - Shows CPU register values in the current frame.
  - Syntax: (gdb) info reg rsp
- info all-reg
  - Prints all general-purpose and floating-point register values.
  - Syntax: (gdb) info all-reg

# Program Stack:

```
3           if (n <= 1)
(gdb) continue
Continuing.

Breakpoint 1, fibonacci (n=4) at fib.c:3
3           if (n <= 1)
(gdb) continue
Continuing.

Breakpoint 1, fibonacci (n=3) at fib.c:3
3           if (n <= 1)
(gdb) bt
#0  fibonacci (n=3) at fib.c:3
#1  0x000055555555551f1 in fibonacci (n=4) at fib.c:6
#2  0x000055555555551f1 in fibonacci (n=5) at fib.c:6
#3  0x000055555555551f1 in fibonacci (n=6) at fib.c:6
#4  0x0000555555555274 in main () at fib.c:17
(gdb) frame 2
#2  0x000055555555551f1 in fibonacci (n=5) at fib.c:6
6               return fibonacci(n - 1) + fibonacci(n - 2);
(gdb) frame 0
#0  fibonacci (n=3) at fib.c:3
3           if (n <= 1)
(gdb) up 1
#1  0x000055555555551f1 in fibonacci (n=4) at fib.c:6
6               return fibonacci(n - 1) + fibonacci(n - 2);
(gdb) bt
#0  fibonacci (n=3) at fib.c:3
#1  0x000055555555551f1 in fibonacci (n=4) at fib.c:6
#2  0x000055555555551f1 in fibonacci (n=5) at fib.c:6
#3  0x000055555555551f1 in fibonacci (n=6) at fib.c:6
#4  0x0000555555555274 in main () at fib.c:17
(gdb) down 1
#0  fibonacci (n=3) at fib.c:3
3           if (n <= 1)
(gdb) info args
n = 3
(gdb) info locals
No locals.
(gdb) info reg
rax            0x3                 3
rbx            0x7fffffffdde8      140737488346600
rcx            0x0                 0
rdx            0x0                 0
rsi            0x2035              8245
rdi            0x3                 3
rbp            0x7fffffffdc10      0x7fffffffdc10
```

```
gs_base        0x0                 0
(gdb) info all-reg
rax            0x3                 3
rbx            0x7fffffffdde8      140737488346600
rcx            0x0                 0
rdx            0x0                 0
rsi            0x2035              8245
rdi            0x3                 3
rbp            0x7fffffffdc10      0x7fffffffdc10
rsp            0x7fffffffdbf0      0x7fffffffdbf0
r8             0x64                100
r9             0x0                 0
r10            0x0                 0
r11            0x0                 0
r12            0x1                 1
r13            0x0                 0
r14            0x555555557da0      93824992247200
r15            0x7ffff7ffd000      140737354125312
rip            0x5555555551d9      0x5555555551d9 <fibonacci+16>
eflags         0x206               [ PF IF ]
cs             0x33                51
ss             0x2b                43
ds             0x0                 0
es             0x0                 0
fs             0x0                 0
gs             0x0                 0
st0            0                   (raw 0x00000000000000000000)
st1            0                   (raw 0x00000000000000000000)
st2            0                   (raw 0x00000000000000000000)
st3            0                   (raw 0x00000000000000000000)
st4            0                   (raw 0x00000000000000000000)
st5            0                   (raw 0x00000000000000000000)
st6            0                   (raw 0x00000000000000000000)
st7            0                   (raw 0x00000000000000000000)
fctrl          0x37f               895
fstat          0x0                 0
ftag           0xffff              65535
fiseg          0x0                 0
fioff          0x0                 0
foseg          0x0                 0
fooff          0x0                 0
fop            0x0                 0
mxcsr          0x1f80              [ IM DM ZM OM UM PM ]
bndcfgu        {raw = 0x0, config = {base = 0x0, reserved = 0x0, prese
d = 0, enabled = 0}}
bndstatus      {raw = 0x0, status = {bde = 0x0, error = 0x0}} {raw = 0
k0             0xc1c1dc1e          3250707486
k1             0x11002021          285220897
--Type <RET> for more, q to quit, c to continue without paging--
```

**Display:**

- print or p
  - Prints the value of an expression.
  - Syntax: print [format] expr
  - Format specifiers:
    - /x → hexadecimal
    - /d → signed decimal
    - /u → unsigned decimal
    - /o → octal
    - /t → binary
    - /a → address (symbolic)
    - /c → character
    - /f → floating point

**Display:**

- call
  - Calls a function (useful for void-returning or side-effect functions).
  - Syntax: call expr
- x (Examine Memory)
  - Syntax: x/[N][u][f] addr
  - N → number of units to examine
  - u → unit size
  - b = byte
  - h = halfword (2 bytes)
  - w = word (4 bytes)
  - g = giant word (8 bytes)
  - f → display format (same as print)
  - addr → memory address to examine

**Display:**

- disassemble or disas
  - Shows machine code instructions for a function or memory range.
  - Syntax: disas [func|address]

```
(gdb) lis
11
12          printf("Enter the number of terms: ");
13          scanf("%d", &n);
14
15          printf("Fibonacci Series:\n");
16          for (i = 0; i < n; i++) {
17              printf("%d ", fibonacci(i));
18          }
19
20          printf("\n");
(gdb) break 15
Breakpoint 1 at 0x1252: file fib.c, line 15.
(gdb) run
Starting program: /home/arindam/fib

This GDB supports auto-downloading debuginfo from th
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-
Enter the number of terms: 5
```

```
Breakpoint 1, main () at fib.c:15
15          printf("Fibonacci Series:\n");
(gdb) p n
$1 = 5
(gdb) p/x n
$2 = 0x5
(gdb) p/t n
$3 = 101
(gdb) p/c n
$4 = 5 '\005'
(gdb) call fibonacci(2)
$5 = 1
(gdb) call fibonacci(4)
$6 = 3
(gdb) p &n
$7 = (int *) 0x7fffffffdcb0
(gdb) x/wx 0x7fffffffdcb0
0x7fffffffdcb0: 0x00000005
(gdb) x/4xb &n
0x7fffffffdcb0: 0x05    0x00    0x00    0x00
```

```
(gdb) disas fibonacci
Dump of assembler code for function fibonacci:
   0x00005555555551c9 <+0>:     endbr64
   0x00005555555551cd <+4>:     push   %rbp
   0x00005555555551ce <+5>:     mov    %rsp,%rbp
   0x00005555555551d1 <+8>:     push   %rbx
   0x00005555555551d2 <+9>:     sub    $0x18,%rsp
   0x00005555555551d6 <+13>:    mov    %edi,-0x14(%rbp)
   0x00005555555551d9 <+16>:    cmpl   $0x1,-0x14(%rbp)
   0x00005555555551dd <+20>:    jg     0x5555555551e4 <fibonacci+27>
   0x00005555555551df <+22>:    mov    -0x14(%rbp),%eax
   0x00005555555551e2 <+25>:    jmp    0x555555555202 <fibonacci+57>
   0x00005555555551e4 <+27>:    mov    -0x14(%rbp),%eax
   0x00005555555551e7 <+30>:    sub    $0x1,%eax
   0x00005555555551ea <+33>:    mov    %eax,%edi
   0x00005555555551ec <+35>:    call   0x5555555551c9 <fibonacci>
   0x00005555555551f1 <+40>:    mov    %eax,%ebx
   0x00005555555551f3 <+42>:    mov    -0x14(%rbp),%eax
   0x00005555555551f6 <+45>:    sub    $0x2,%eax
   0x00005555555551f9 <+48>:    mov    %eax,%edi
   0x00005555555551fb <+50>:    call   0x5555555551c9 <fibonacci>
   0x0000555555555200 <+55>:    add    %ebx,%eax
   0x0000555555555202 <+57>:    mov    -0x8(%rbp),%rbx
   0x0000555555555206 <+61>:    leave
   0x0000555555555207 <+62>:    ret
End of assembler dump.
(gdb) disas main
Dump of assembler code for function main:
   0x0000555555555208 <+0>:     endbr64
   0x000055555555520c <+4>:     push   %rbp
   0x000055555555520d <+5>:     mov    %rsp,%rbp
   0x0000555555555210 <+8>:     sub    $0x10,%rsp
   0x0000555555555214 <+12>:    mov    %fs:0x28,%rax
   0x000055555555521d <+21>:    mov    %rax,-0x8(%rbp)
   0x0000555555555221 <+25>:    xor    %eax,%eax
   0x0000555555555223 <+27>:    lea    0xdda(%rip),%rax        # 0x555555556004
   0x000055555555522a <+34>:    mov    %rax,%rdi
   0x000055555555522d <+37>:    mov    $0x0,%eax
```

**Automatic Display:**

- display
  - Automatically shows the value of an expression every time execution stops.
  - Syntax: (gdb) display [/f] expression
  - /f — optional format (same as in print, like /x, /d, /t, etc.)
  - expression — any valid C expression (variable, function call, pointer dereference, etc.)
- display (with no arguments)
  - Shows all currently active automatic display expressions.
  - Syntax: (gdb) display
- undisplay
  - Removes one or more expressions from the display list.
  - Syntax: undisplay n [n is the number shown in the display list]

**Automatic Display:**

- disable display
  - Disables automatic display of a particular expression without removing it.
  - Syntax: (gdb) disable display n [n is the number shown in the display list]
- enable display
  - Re-enables a previously disabled automatic display.
  - Syntax: enable display n [n is the number shown in the display list]
- info display
  - Lists all currently defined automatic display expressions, including disabled ones.
  - Syntax: (gdb) info display

# Automatic Display:

```
15          printf("Fibonacci Series:\n");
16          for (i = 0; i < n; i++) {
17              printf("%d ", fibonacci(i));
18          }
19
20          printf("\n");
(gdb) break 16
Breakpoint 3 at 0x1261: file fib.c, line 16.
(gdb) run
Starting program: /home/arindam/fib
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter the number of terms: 5
Fibonacci Series:

Breakpoint 3, main () at fib.c:16
16          for (i = 0; i < n; i++) {
(gdb) display i
1: i = 32767
(gdb) next
17              printf("%d ", fibonacci(i));
1: i = 0
(gdb) next
16          for (i = 0; i < n; i++) {
1: i = 0
(gdb) next
17              printf("%d ", fibonacci(i));
1: i = 1
(gdb) next
16          for (i = 0; i < n; i++) {
1: i = 1
(gdb) next
17              printf("%d ", fibonacci(i));
1: i = 2
(gdb) display
1: i = 2
(gdb)
```

```
Enter the number of terms: 5
Fibonacci Series:

Breakpoint 3, main () at fib.c:16
16          for (i = 0; i < n; i++) {
(gdb) display i
3: i = 32767
(gdb) next
17              printf("%d ", fibonacci(i));
3: i = 0
(gdb) next
16          for (i = 0; i < n; i++) {
3: i = 0
(gdb) next
17              printf("%d ", fibonacci(i));
3: i = 1
(gdb) diplay n
Undefined command: "diplay".  Try "help".
(gdb) display n
4: n = 5
(gdb) display
3: i = 1
4: n = 5
(gdb) disable display 4
(gdb) next
16          for (i = 0; i < n; i++) {
3: i = 1
(gdb) enable display 4
(gdb) next
17              printf("%d ", fibonacci(i));
3: i = 2
4: n = 5
(gdb) info display
Auto-display expressions now in effect:
Num Enb Expression
3:   y  i
4:   y  n
(gdb)
```

**Expressions:**

- Standard C Expressions
  - You can use any valid C/C++ expression in GDB.
  - Example: (gdb) print 5 + 2
    $1 = 7
    (gdb) print fibonacci(4)
    $2 = 3
    (gdb) print n
    $3 = 5
- Memory Block Access (addr @ len)
  - Reads a block of memory (interpreted as an array).
  - Example: (gdb) set $arr = &n  # Suppose n is defined as int n;
    (gdb) x/4dw $arr
    0x7fffffffe4ac: 5  0  0  0
    (gdb) print $arr @ 4  # Interprets the memory as an array of 4 ints
    $4 = {5, 0, 0, 0}

**Expressions:**

- File-qualified Names (file::name)
  - Refers to a symbol defined in a specific file.
  - Example: (gdb) print main::n (Useful when multiple files define symbols with the same name.)
- Typed Memory Access ({type}addr)
  - Reads memory at an address using a specified type.
  - Example: (gdb) print {int}0x7fffffffe4ac
    $5 = 5 #This reads the memory at address 0x7fffffffe4ac and interprets it as an int.
- 5. Special $ Expressions

| Symbol | Meaning | Example |
|---|---|---|
| $ | Most recent printed value | print $ + 2 (if $ = 5) |
| $n | The *n*th printed value (e.g., $1, $2) | print $1 * 3 |
| $$ | Value before the last one | print $$ |
| $$n | *n* steps before the most recent $ | print $$2 |
| $_ | Last address examined with x | x/i $_ |
| $__ | Value at the last examined address | print $__ |

**Expressions:**

- Convenience Variables ($var)
  - These are user-defined variables you can create and use in GDB.
  - Example: (gdb) set $a = 10
    (gdb) print $a * 2
    $6 = 20
    (gdb) set $fib = fibonacci(5)
    (gdb) print $fib
    $7 = 5

**Symbol Table Inspection:**

- info address <symbol>
  - Shows where a symbol (variable or function) is stored in memory.
  - Example: (gdb) info address fibonacci

    Symbol "fibonacci" is static storage at address 0x555555555166.
- info func [regex]
  - Lists all functions defined in your program or those matching a regex pattern.
  - Example: (gdb) info func

    All defined functions:

    File fib.c:

    int fibonacci(int);

    int main();

    (gdb) info func fib.* #Lists only functions starting with fib.

**Symbol Table Inspection:**

- info var [regex]
  - Lists global variables, optionally filtered with a regex.
  - Example: (gdb) info var
    All defined variables:
    File fib.c:
    int n;
    int i;
- Type Inspection Commands
  - Displays the data type of an expression, variable, or value.
  - Syntax: whatis <expr>
  - Example: (gdb) whatis n
    type = int
    (gdb) whatis fibonacci
    type = int (int)

**Symbol Table Inspection:**

- ptype <expr>
  - Gives a detailed description of a data type or structure.
  - Example: (gdb) ptype fibonacci
    type = int (int)
    (gdb) ptype n
    type = int
- ptype <type>
  - Shows the structure/definition of a user-defined type.
  - Example: struct Point { int x; int y; };
    (gdb) ptype struct Point
    type = struct Point {
    int x;
    int y;
    }

**GDB Scripts:**

- source <script>
  - Run a list of GDB commands from a file.
  - Create a script file called my_gdb_script.gdb:

    echo "break main" > my_gdb_script.gdb

    echo "run" >> my_gdb_script.gdb
  - Then in GDB: (gdb) source my_gdb_script.gdb
    - This sets a breakpoint at main() and starts execution automatically.
- define <cmd> ... end
  - Define a custom command in GDB by grouping multiple GDB commands.
  - Example: (gdb) define hello

    > echo Hello from GDB!\n

    > info registers

    > end
  - Now when you type: (gdb) hello
    - It prints "Hello from GDB!" and then shows the CPU registers.

**GDB Scripts:**

- document <cmd> ... end
  - Add help text for a custom command you've defined using define.
  - Example: (gdb) document hello

    > Prints hello and shows register state.

    > end
    - Now if you type: (gdb) help hello
    - You'll see: Prints hello and shows register state.

**GDB Signal Handling:**

| Command | Purpose | Syntax | Example |
|---|---|---|---|
| handle | Control how GDB responds to signals | handle <signal> [actions] | handle SIGINT nostop noprint nopass |
| info signals | Show GDB’s current actions for all signals | info signals | info signals |

**Signal Handling Actions:**

| Action | Purpose | Syntax | Example |
|---|---|---|---|
| print | Print a message when the signal is received | handle <signal> print | handle SIGTERM print |
| noprint | Do not print any message when signal is received | handle <signal> noprint | handle SIGINT noprint |
| stop | Stop program execution when signal is received | handle <signal> stop | handle SIGFPE stop |
| nostop | Do not stop program execution on signal | handle <signal> nostop | handle SIGUSR1 nostop |
| pass | Let program handle the signal | handle <signal> pass | handle SIGUSR1 pass |
| nopass | Prevent program from handling the signal | handle <signal> nopass | handle SIGSEGV nopass |

**Use Cases:**

| Command | Purpose | Syntax | Example |
|---|---|---|---|
| handle SIGINT nostop noprint nopass | Ignore Ctrl+C, don't stop, print, or pass to program | handle SIGINT [actions] | handle SIGINT nostop noprint nopass |
| handle SIGSEGV stop print nopass | Catch segfaults, stop and print message, don’t let program handle it | handle SIGSEGV [actions] | handle SIGSEGV stop print nopass |
| handle SIGUSR1 nostop pass noprint | Allow program to handle SIGUSR1 silently without stopping GDB | handle SIGUSR1 [actions] | handle SIGUSR1 nostop pass noprint |
| info signals | View how GDB handles all signals | info signals | info signals |

**Debugging Targets:**

| Command | Purpose | Syntax | Example |
|---|---|---|---|
| target | Connect to a target (remote system, file, device, etc.) | target <type> <params> | target remote localhost:1234 |
| help target | Display available target types supported by GDB | help target | help target |
| attach | Attach GDB to a running process | attach <PID> | attach 12345 |
| detach | Detach from the currently attached process or target | detach | detach |

**Debugging Targets Use Cases:**

| Command | Purpose | Syntax | Example |
|---|---|---|---|
| target exec ./program | Load a local executable as the target | target exec <filename> | target exec ./fib |
| target remote localhost:1234 | Connect to a remote debugging stub (e.g. QEMU or GDBServer) | target remote <host>:<port> | target remote 192.168.1.100:1234 |
| attach 5678 | Attach to a running process with PID 5678 | attach <PID> | attach 5678 |
| detach | Detach from the target process | detach | detach |
| help target | View available target types supported by GDB | help target | help target |

**Controlling GDB:**

| Command | Purpose | Syntax | Example |
|---|---|---|---|
| set param value | Set a GDB internal parameter | set <param> <value> | set confirm off |
| show param | Show current value of a GDB internal parameter | show <param> | show confirm |
| show commands | Show last 10 GDB commands used | show commands | show commands |
| show commands n | Show commands around a certain index | show commands <n> | show commands 15 |
| show commands + | Show next 10 commands from the last listing | show commands + | show commands + |

# Parameters and Groups with Examples:

| Parameter | Purpose | Set Example | Show Example |
|---|---|---|---|
| confirm | Enable/disable prompts for confirmation | set confirm off | show confirm |
| editing | Enable/disable command-line editing | set editing on | show editing |
| height | Set number of lines before paginating output | set height 0 (disable pause) | show height |
| language | Set the language for expressions | set language c | show language |
| listsize | Number of source lines list command shows | set listsize 20 | show listsize |
| prompt | Change GDB prompt string | set prompt (gdb-custom) | show prompt |
| radix | Set default base (input/output radix) | set radix 16 | show radix |
| verbose | Enable detailed output when loading symbols | set verbose on | show verbose |
| width | Set line width before wrapping output | set width 100 | show width |
| write | Allow binary/core file modification | set write on | show write |

**History:**

| Command | Purpose | Set Example | Show Example |
|---|---|---|---|
| history expansion | Enable/disable history expansion | set history expansion off | show history expansion |
| history file | File to store command history | set history filename .gdb_hist | show history file |
| history size | Number of history entries to keep | set history size 100 | show history size |
| history save | Save history to file | set history save on | show history save |

# Print:

| Command | Purpose | Set Example | Show Example |
|---|---|---|---|
| print address | Show memory addresses in output | set print address on | show print address |
| print array | Compact or pretty array output | set print array on | show print array |
| print demangle | Display demangled C++ names | set print demangle on | show print demangle |
| print asm-demangle | Demangle in disassembly output | set print asm-demangle on | show print asm-demangle |
| print elements | Limit number of array elements printed | set print elements 5 | show print elements |
| print object | Show derived class types for C++ objects | set print object on | show print object |
| print pretty | Pretty-print structs (indentation) | set print pretty on | show print pretty |
| print union | Show union members in structs | set print union on | show print union |
| print vtbl | Show virtual function tables in C++ objects | set print vtbl on | show print vtbl |




**References:** https://users.ece.utexas.edu/~adnan/gdb-refcard.pdf

**Next Topic : Lab Session (GDB over different programs)**